

Relationen, Algebra, Diskrete Mathematik

Teil 5 - Graphen

Martin Zaefferer

Graphen

Quellen (dieser Folien)

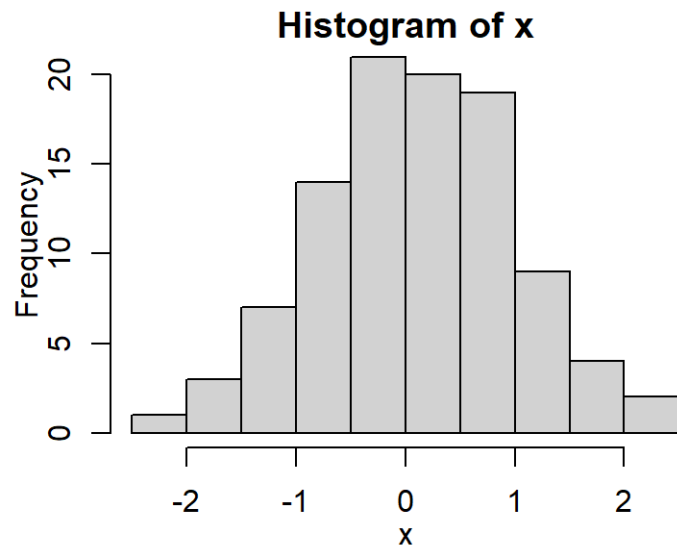
- Mit Änderungen / Ergänzungen, basierend auf:
 - Titel: TI3110TU - Algorithms and Data Structures; Source code for the lecture notes for the TU Delft Algorithms and Data Structures minor course.
 - Autoren: Georgios Gousios, Joseph Hejderup, Maria Kechagia, Mozhan Soltani, Pavel Kucherbaev, Joris Quist, Daan Middendorp, Wouter Zorgdrager
 - Link: <https://github.com/gousiosg/algo-ds>

Grundlagen

Graphen

- Häufig äquivalent: Netze, Netzwerke
- Nicht zu verwechseln mit:

Graphen (Diagramme)



Grafen (Adelstitel)



Graphen (Kohlenstoffstruktur)



Graphen

- Graphen ohne Gewichte $G = (V, E)$
 - V : Menge von Knoten (engl. *nodes* oder *vertices*)
 - E : Menge von Kanten (engl. *edges* oder *links*) oder Verbindungen
- Graphen mit Gewichten $G = (V, E, W)$
 - W : Menge von Kantengewichten (engl. *weights*)

Knoten stellen Objekte dar

- z.B.
 - Menschen
 - Städte
 - Computer
 - Neuronen

Kanten stellen Beziehungen dar

- z.B.
 - Menschen: Verwandtschaft, Freundschaft
 - Städte: Verkehrsverbindungen
 - Computer: Netzwerkverbindungen
 - Neuronen: Synapsen

Kanten (Beziehungen) können eine Richtung haben

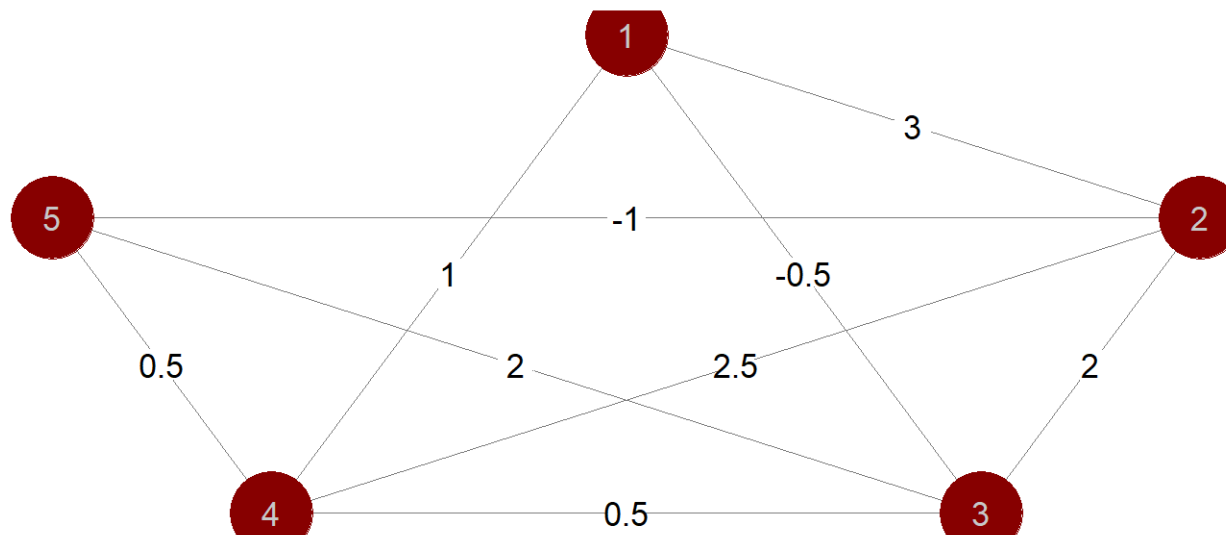
- z.B.
 - Menschen: Verwandtschaft - A ist Kind von B
 - Städte: Verkehrsverbindungen - Einbahnstraße
 - Computer: Hierarchische Netzwerkverbindungen
 - Neuronen: Synapsen

Position von Knoten

- Position der Knoten (in grafischer Darstellung) verändert grundsätzlich nicht die Bedeutung des Graphen
- Kann aber bei bestimmten Anwendungen von Bedeutung sein
 - z.B. wenn Knotenpositionen auf einer Karte verortet werden, z.B. Städte

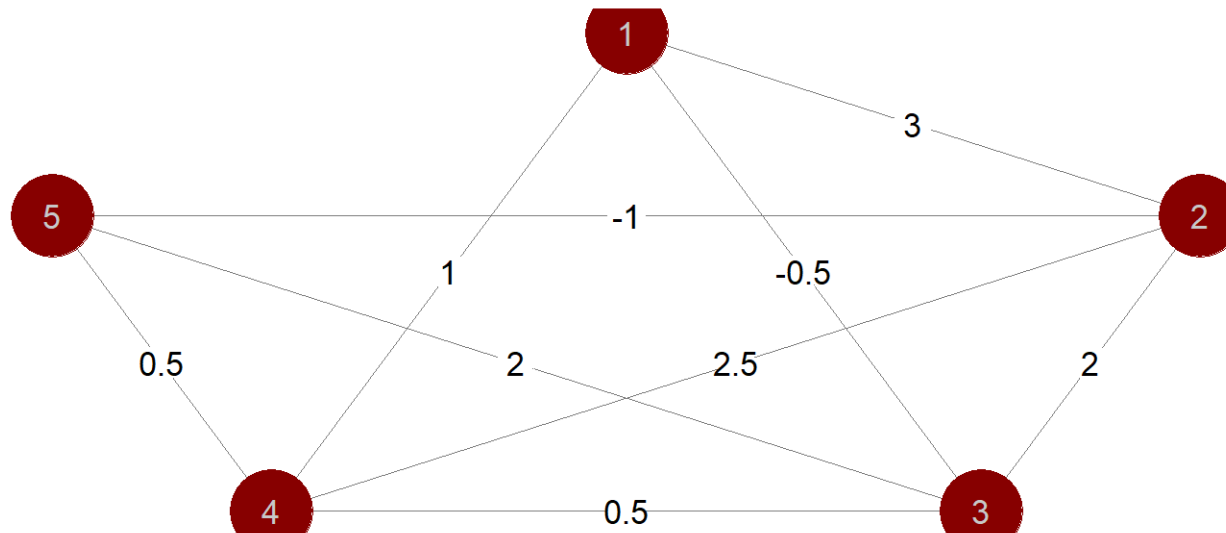
Gewichte auf Kanten

z.B. Reisekosten, Entfernung, Wahrscheinlichkeit eines Wechsels. In einem gerichteten Graphen können die Kosten in einer Richtung anders sein als in der Gegenrichtung (z.B. bergauf und bergab).

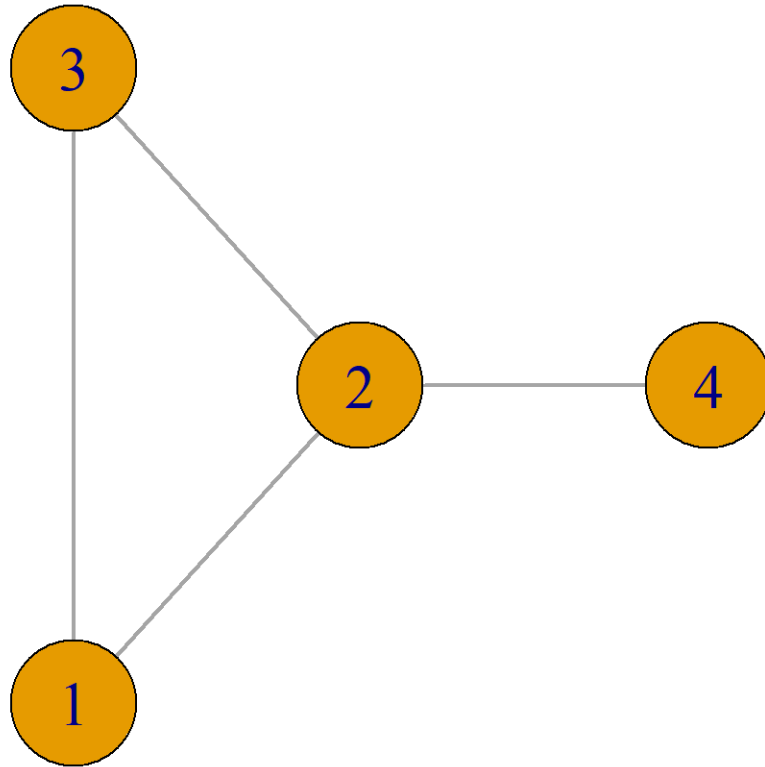


Knotengrad

Anzahl der angrenzenden Kanten eines Knoten. Hier: $\text{Grad}("1") = 3$; $\text{Grad}("3") = 4$.
Was ist Ihr Knotengrad auf LinkedIn?

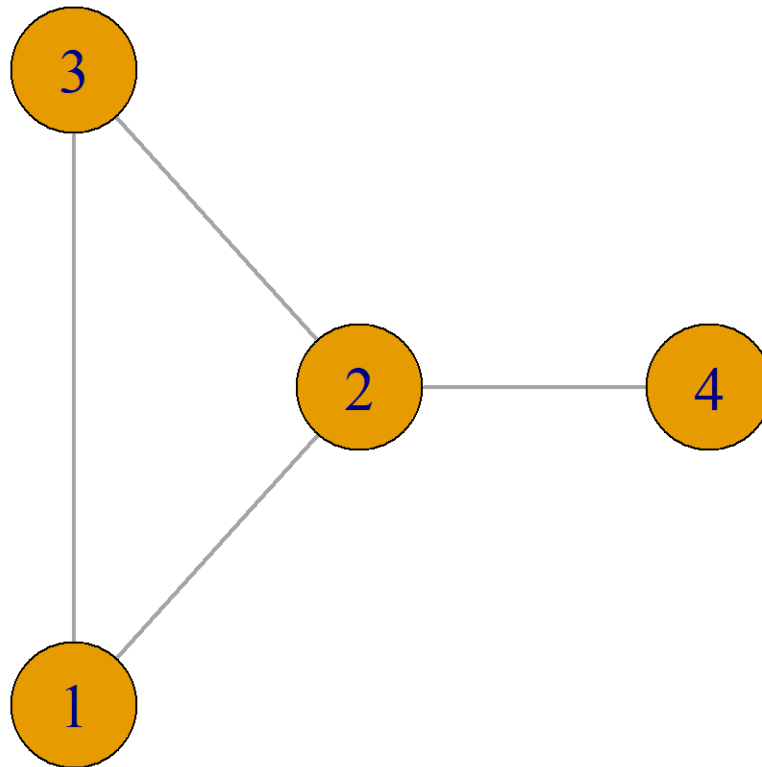


Darstellungsform: Zeichnung



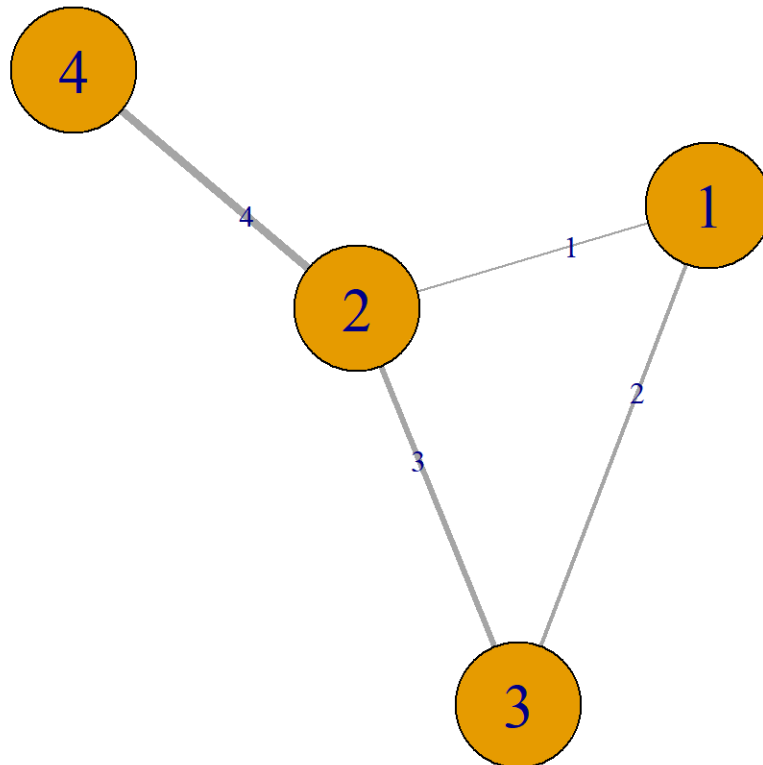
Graphen: Ungerichteter Graph

- Verbindungen haben keine Richtung



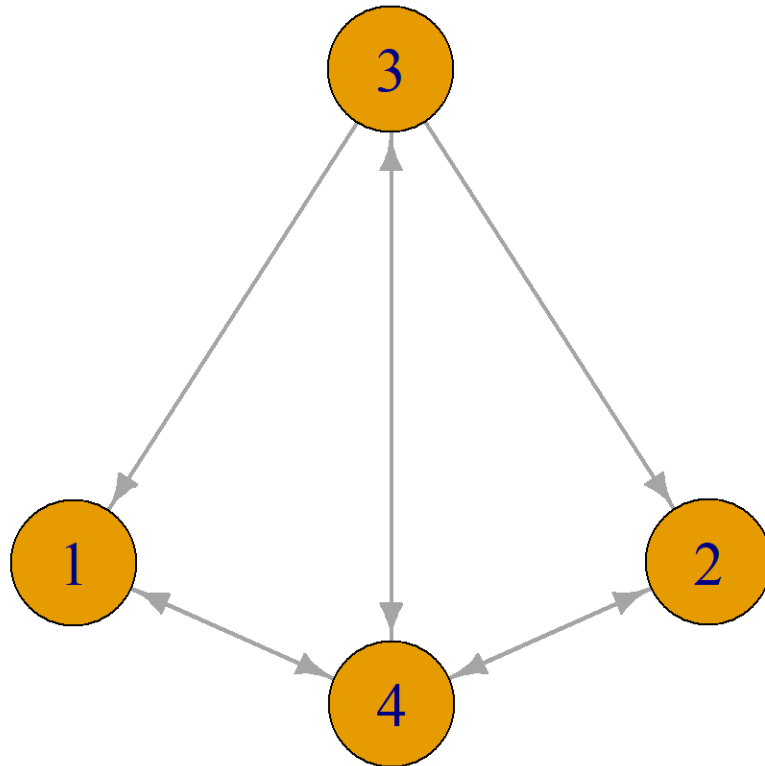
Graphen: mit Gewichten

- Ungerichteter Graph, mit Gewichten



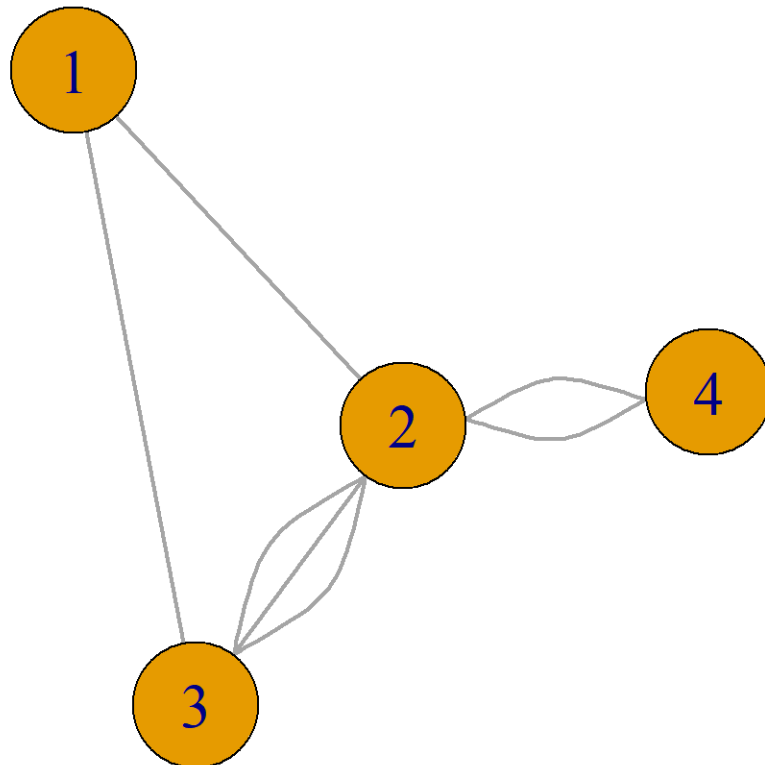
Graphen: Gerichteter Graph

- Verbindungen haben eine Richtung



Multigraph

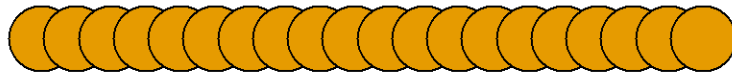
- Mehrere Verbindungen zwischen zwei Knoten möglich (hier: ungerichtet)



Typen von Graphen: Leerer Graph

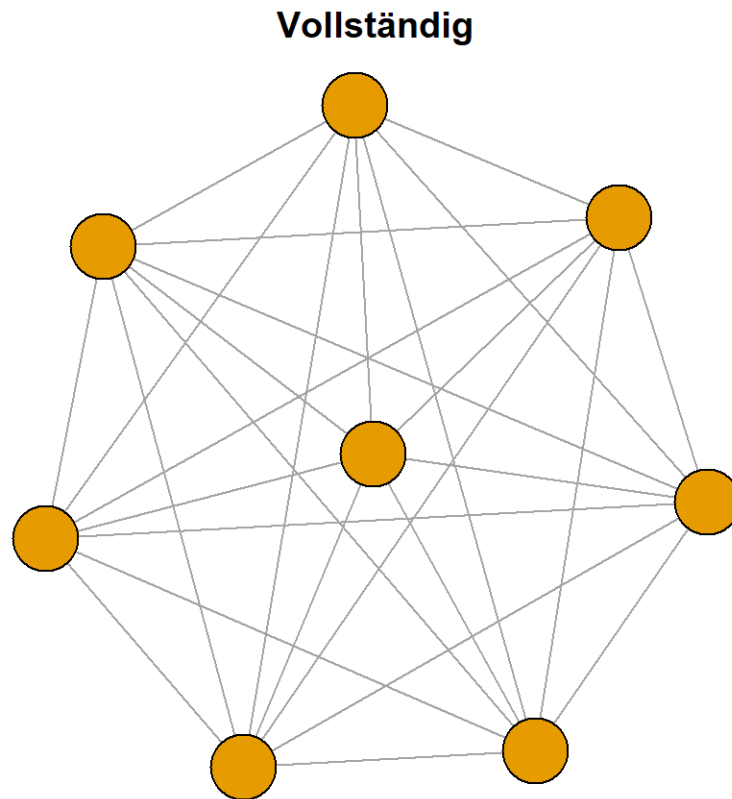
- Keine Kanten (aber: Knoten!)

Leer / Null



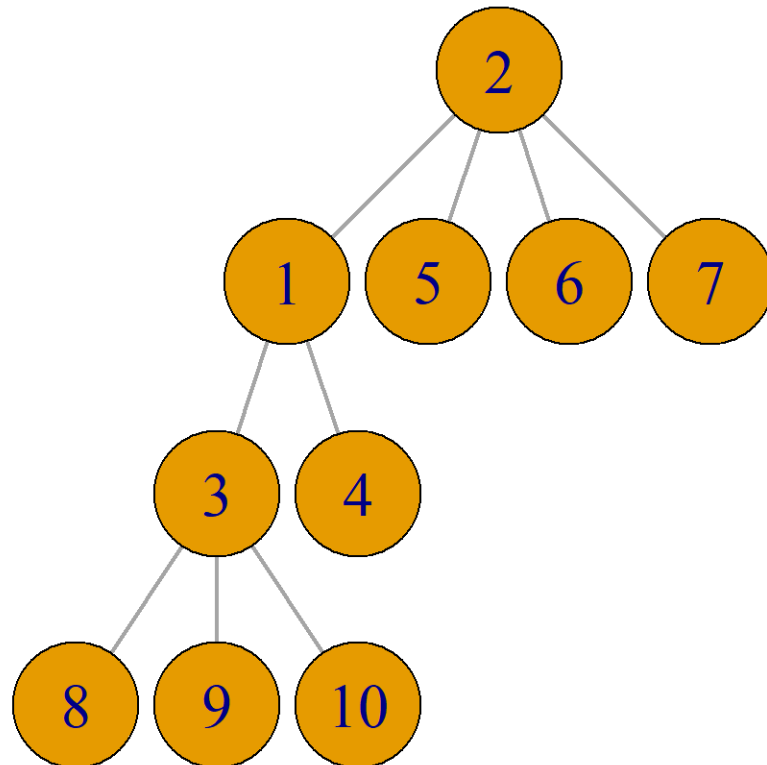
Typen von Graphen: Vollständiger Graph

- Kanten zwischen allen Knoten



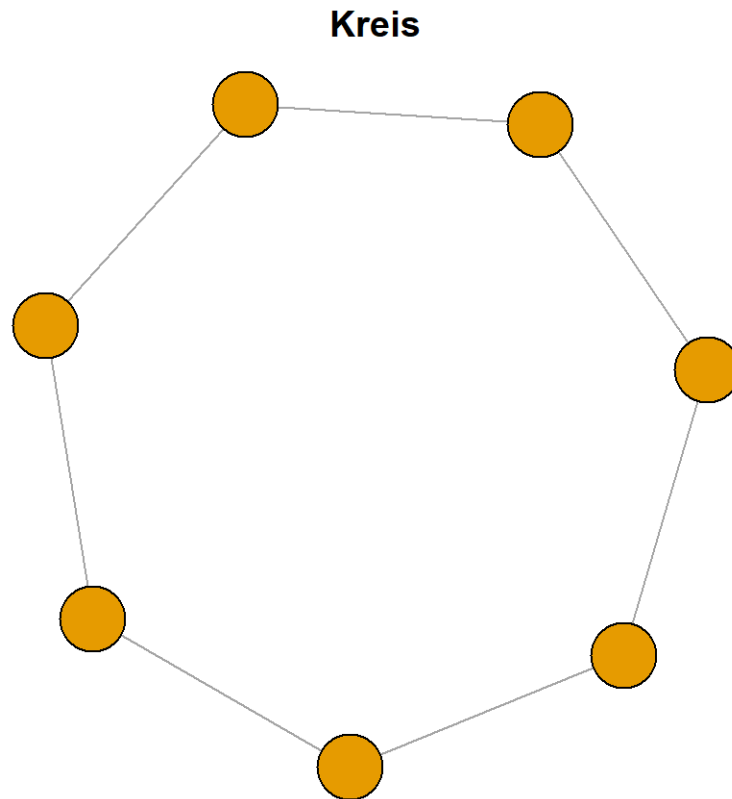
Typen von Graphen: Baum

- Graphen ohne geschlossene Pfade / Zyklen



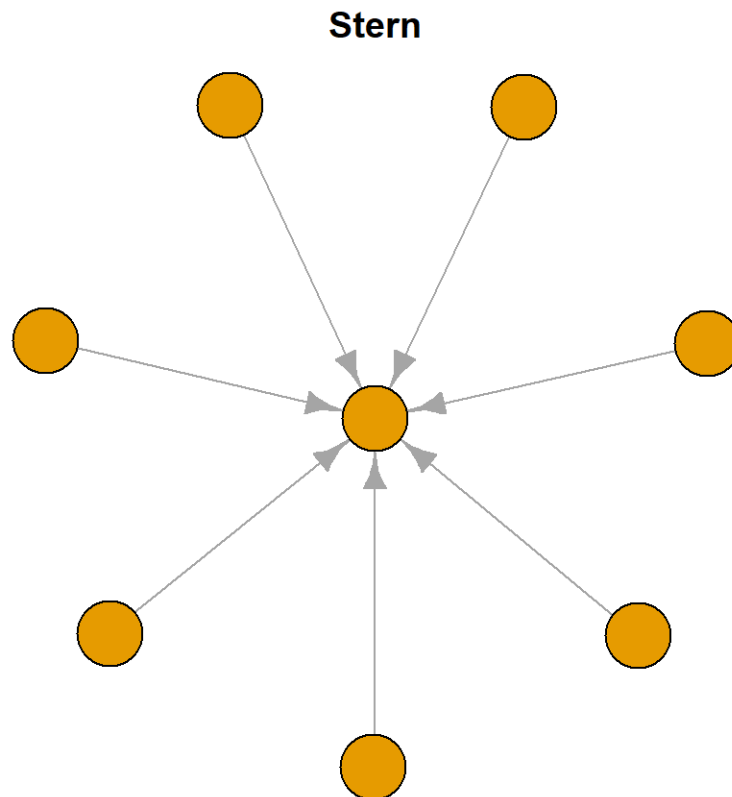
Typen von Graphen: Kreis

- Gleiche Anzahl Kanten / Knoten. Alle Knoten miteinander verbunden.
- Jeder Knoten hat Knotengrad 2



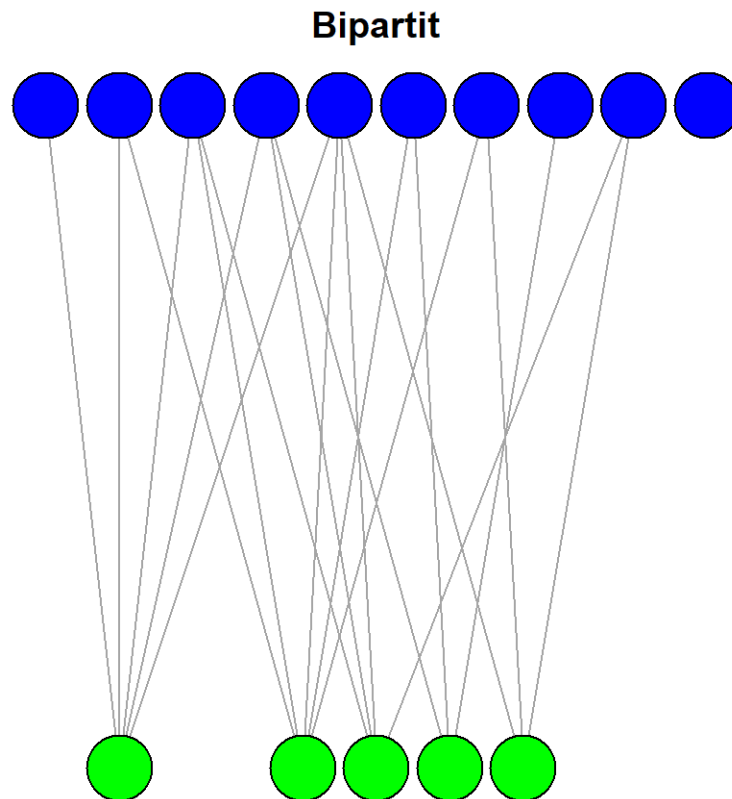
Typen von Graphen: Stern

- Ein Knoten ist mit allen anderen verbunden



Typen von Graphen: Bipartiter Graph

- Zwei Knotenmengen, Verbindungen nur zwischen den Mengen, nicht innerhalb einer Menge

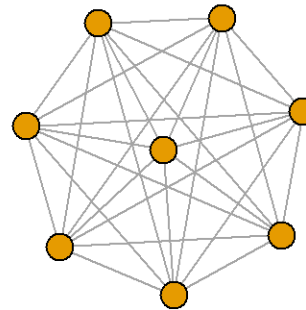


Arten von Graphen: Übersicht

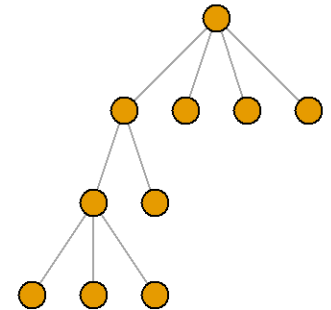
Leer / Null



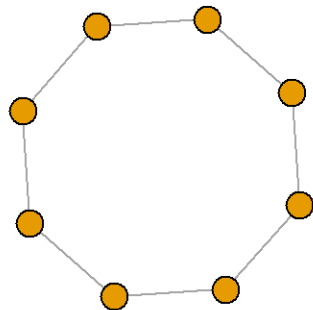
Vollständig



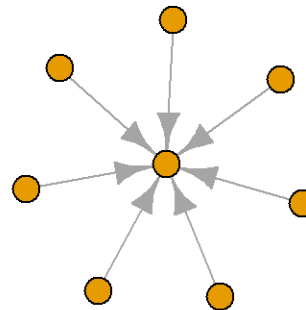
Baum



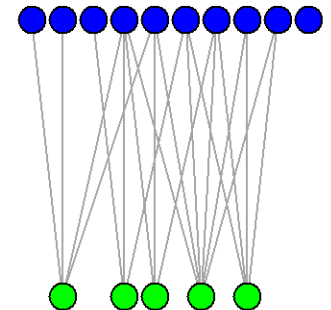
Kreis



Stern



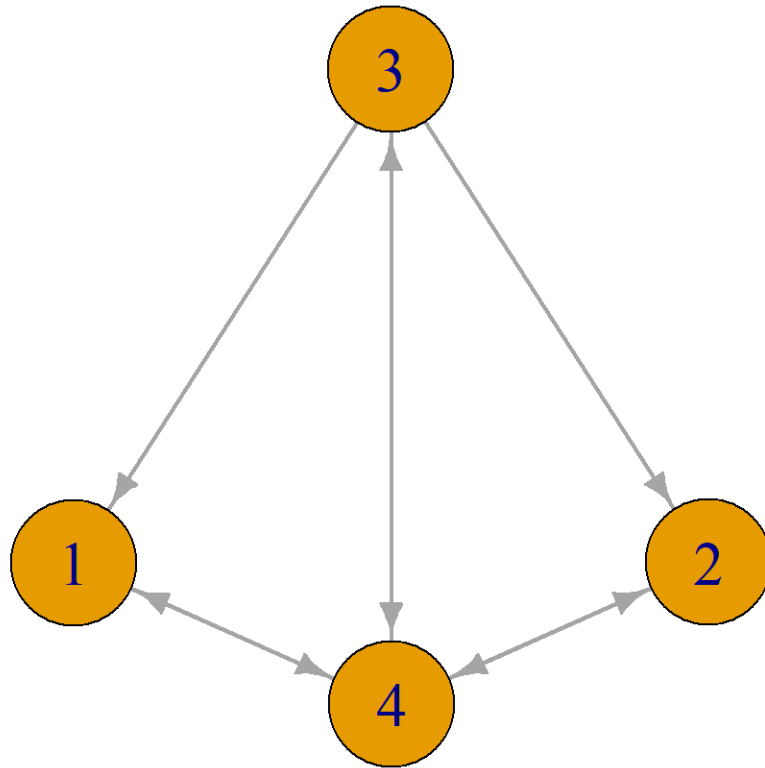
Bipartit



Darstellungsform: Adjazenzmatrix

- Bei n Knoten: $n \times n$ Adjazenzmatrix A
- Jeder Knoten entspricht einer Zeile i / Spalte j
- Wenn zwei Knoten mit Kanten verbunden sind: Eintrag in der entsprechenden Zelle $a_{ij} \neq 0$
 - Wenn keine Gewichte vorliegen: a_{ij} ist 0 oder 1
 - Wenn Gewichte vorliegen: a_{ij} ist 0 oder Gewicht
 - Bei Multigraphen
 - ohne Gewichte: a_{ij} ist Anzahl Kanten
 - mit Gewichten: andere Darstellungsform erforderlich (zumindest ergänzend)
- Ungerichteter Graph $\rightarrow$ Adjazenzmatrix ist immer symmetrisch
- Gerichteter Graph $\rightarrow$ Adjazenzmatrix kann asymmetrisch sein

Darstellungsform: Adjazenzmatrix

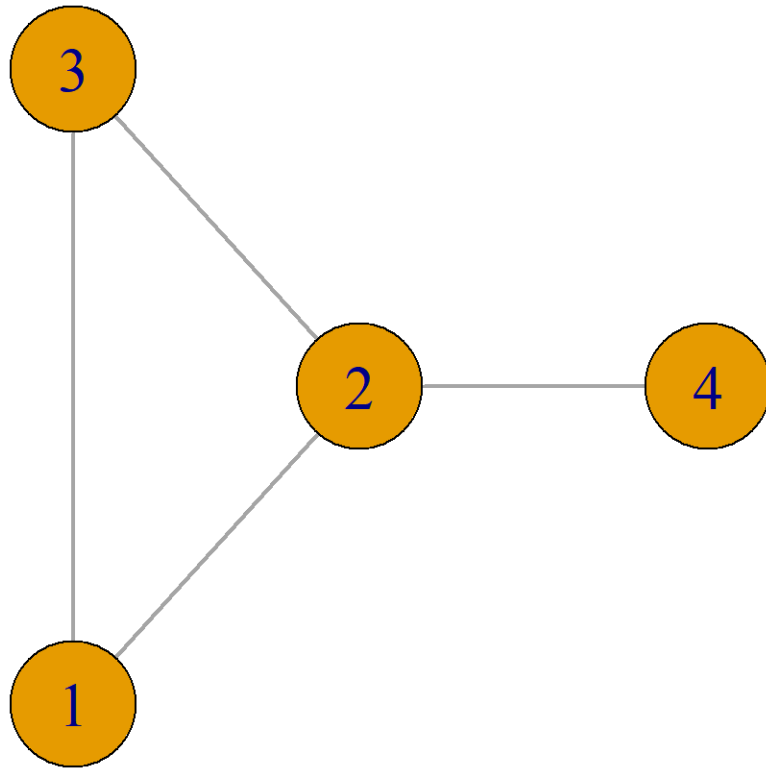


$$\begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

Darstellungsform: Adjazenzliste

- Ungerichtet: Auflistung aller benachbarten Knoten
 - Nachbar: Verbindung über einzelne Kante
- Gerichtet: 2 Listen möglich (alle Vorgänger jedes Knotens & alle Nachfolger jedes Knotens)
- Anmerkung: Genaue Notation der Liste variiert in der Literatur stark
- Nützlich für "spärliche Graphen" (mit wenigen Kanten)

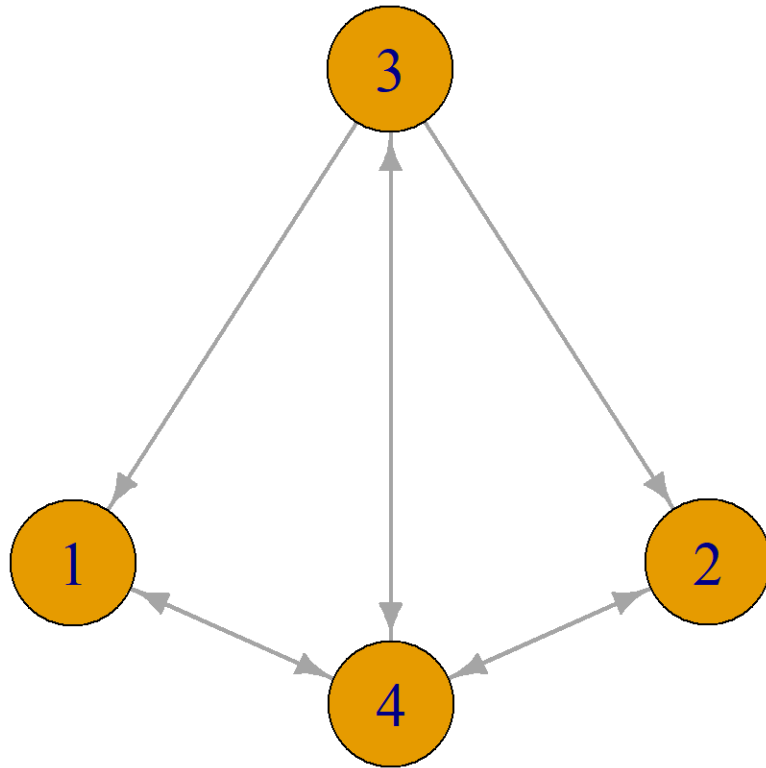
Darstellungsform: Adjazenzliste



Nachbarn

- 1 – 2, 3
- 2 – 1, 3, 4
- 3 – 1, 2
- 4 – 2

Darstellungsform: Adjazenzliste



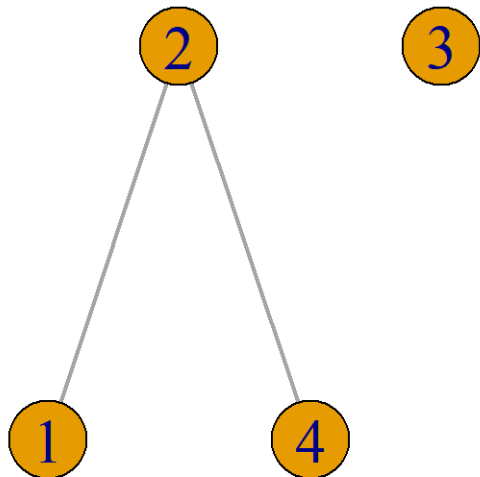
Nachfolger Vorgänger

- 1 -> 4 • 1 <- 3,4
- 2 -> 4 • 2 <- 3,4
- 3 -> 1, 2, 4 • 3 <- 4
- 4 -> 1, 2, 3 • 4 <- 1, 2, 3

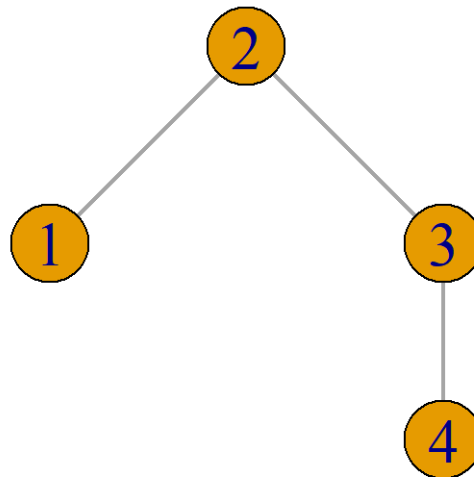
Pfad

- Ein Pfad ist eine Sequenz von Kanten die eine Sequenz von *unterschiedlichen* Knoten verbinden. Knoten kommen nur ein mal je Sequenz vor.

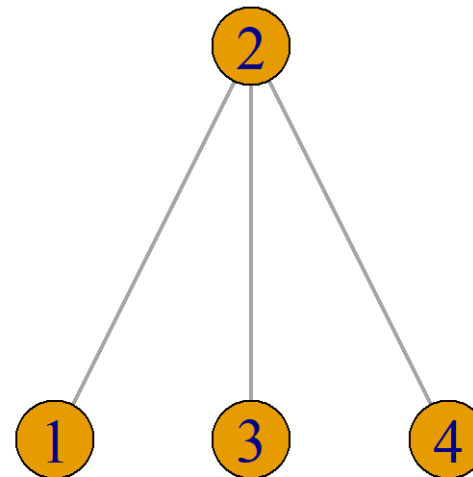
Pfad enthalten



Pfad (gesamt)



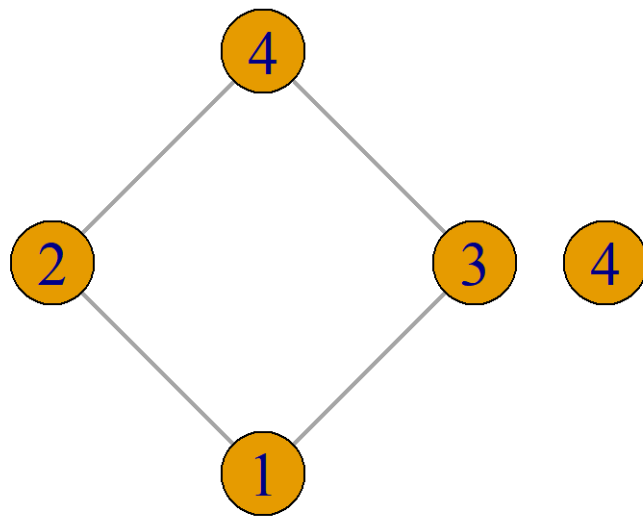
Pfad enthalten



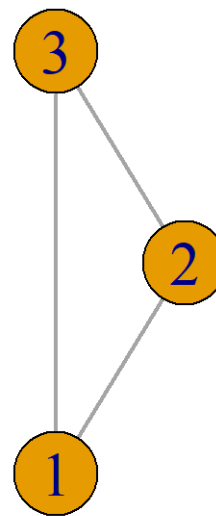
Zyklus

Ein geschlossener Pfad, bei dem alle Kanten unterschiedlich sind (0 - 1 - 3 - 2 - 0).
Bäume können keine Zyklen haben.

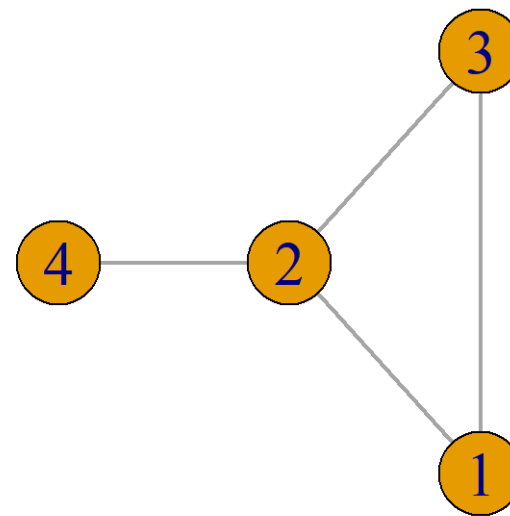
Zyklus (gesamt)



Zyklus enthalten



Zyklus enthalten



Hamiltonscher Pfad / Zyklus

- Hamiltonscher Pfad: Pfad, der *jeden Knoten* genau einmal enthält (0 - 2 - 3 - 1)
 - Beispiel: Ein Tourist besucht alle Sehenswürdigkeiten der Stadt genau einmal
- Hamiltonscher Zyklus: Zyklus, der *jeden Knoten* genau einmal enthält: (0 - 2 - 3 - 1 - 0)
 - Beispiel: Ein Tourist besucht alle Sehenswürdigkeiten der Stadt und kehrt zum Bahnhof zurück, an dem er seine Reise begonnen hat.

Eulerscher Pfad / Zyklus

- Eulerscher Pfad: Pfad, der *jede Kante* genau einmal enthält (2 - 3 - 1 - 0 - 2 - 1)
 - Beispiel: ein Tourist, durch jede Straße einer Stadt läuft, ohne eine Straße zweimal zu nutzen.
- Eulerscher Zyklus: Zyklus, der jede *Kante* genau einmal enthält z.B.: (0 - 1 - 2 - 0 - 3 - 1 - 4 - 2 - 3 - 4 - 0)
 - Beispiel: ein Tourist, durch jede Straße einer Stadt läuft und zum Bahnhof zurückkehrt, ohne eine Straße zweimal zu nutzen.

Übungsaufgaben

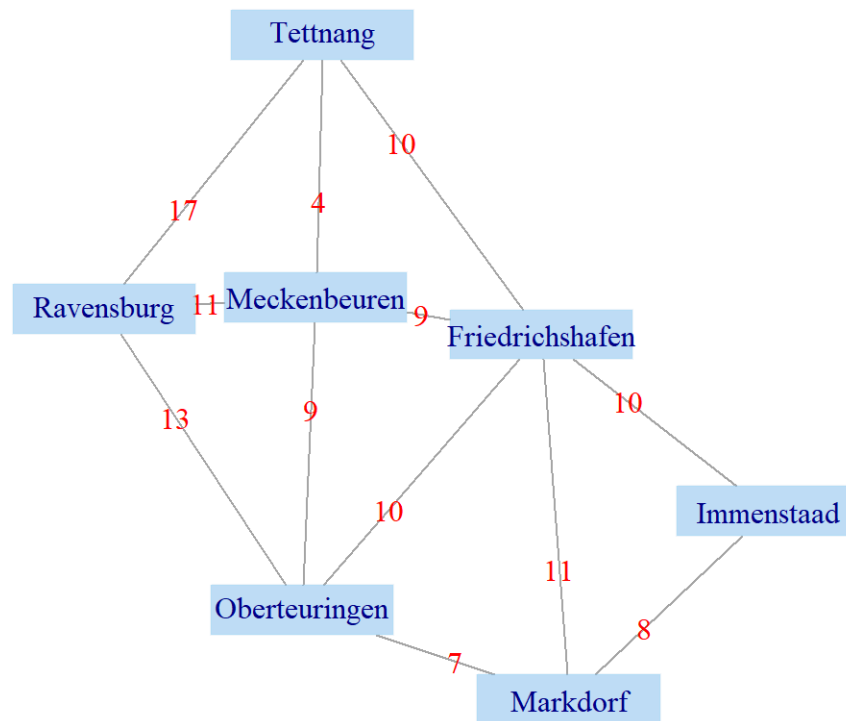
- 7.1 - 7.5 (Übung im moodle Kurse)

Kürzester Pfad

Kürzester Pfad

- Eine wichtige Fragestellung zu Graphen:
 - Was ist der kürzeste Weg zwischen zwei Orten?
 - Oder statt “kürzester Weg” auch “geringste Kosten”
- In Graph-Sprache: Welcher Pfad zwischen zwei Knoten hat die geringste Summe von Gewichten?
- Hier werden immer zwei Dinge als Graph dargestellt
 - **Problemstellung**: Knoten bzw. Orte, Kanten bzw. Verbindungsmöglichkeiten, und Gewichten bzw. Distanzen, beliebiger Typ
 - **Lösungskandidaten**: Untermenge der Problemstellungsknoten / -kanten, Typ Pfad

Beispiel: Ravensburg → Immenstaad



Frage:

- Welche Lösungsstrategie würden Sie wählen?

Beispiel Lösung: Brute Force

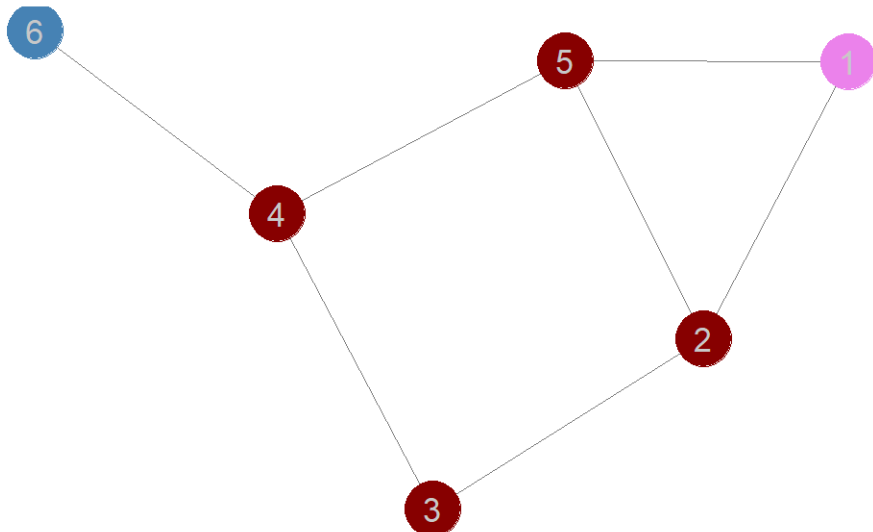
- Alle möglichen Pfade ausprobieren
- Liefert garantiert die optimale Lösung
- Aber ineffizient
 - Hohe Zahl von Möglichkeiten.
 - Hier: mit nur 7 Knoten und 12 Kanten: 29 Pfade von "Ra" nach "Im"

Greedy Search / best-first

- Jeweils die nächste zulässige Kante mit dem geringsten Gewicht
 - Zulässig: führt zu Knoten der noch nicht besucht wurde
 - hier: Ra - Me - Te - Fr - Im
 - Nicht besonders gut: 35 km Pfadlänge
- Häufig keine gute Lösung
- Kann in "Sackgassen" führen

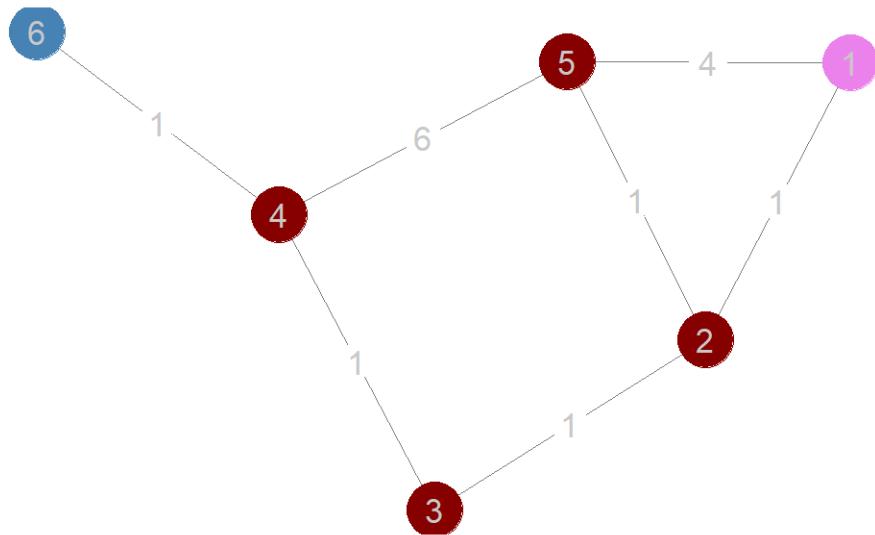
Kürzester Pfad: Einheitliche Gewichte

- Welcher Pfad ist hier der kürzeste (überall gleiche Gewichte), von Knoten 1 zu 6?



Kürzester Pfad

- Was ist jetzt der kürzeste Pfad?
- Wie kann man programmatisch den kürzesten Weg zwischen 2 beliebigen Knoten für einen beliebigen Graphen berechnen?



Kürzester Pfad mit Dijkstra Algorithmus

Dijkstra - Initialisierung

- Graph G mit Knoten $G.V$, ein Startknoten $s \in G.V$
- Distanz d , Status π , Vorgängerknoten α

initialize(G, s)

for each vertex $v \in G.V$

$v.d = \infty$

$v.\pi = \text{unvisited}$

$v.\alpha = \text{none}$

$s.d = 0$

Dijkstra - Update

- Aktueller Knoten u , Nachbarknoten v , Gewicht zwischen Knoten $w(u, v)$
- Distanz d , Status π , Vorgängerknoten α

update(u, v, w)

if $v.d > u.d + w(u, v)$

$v.d = u.d + w(u, v)$

$v.\alpha = u$

Dijkstra - Algorithmus

- Graph G mit Knoten $G.V$, ein Startknoten $s \in G.V$
- Distanz d , Status π , Vorgängerknoten α
- Aktueller Knoten u

```
Dijkstra( $G, w, s$ )  
  initialize( $G, s$ )  
   $u = s$   
  while any (or the target) node  $v. \pi == \text{unvisited}$   
     $u. \pi = \text{visited}$   
    for each neighboring node  $v$  of  $u$   
      if  $v. \pi == \text{unvisited}$   
        update( $u, v, w$ )  
   $u = \text{find unvisited node with smallest } u. d$ 
```

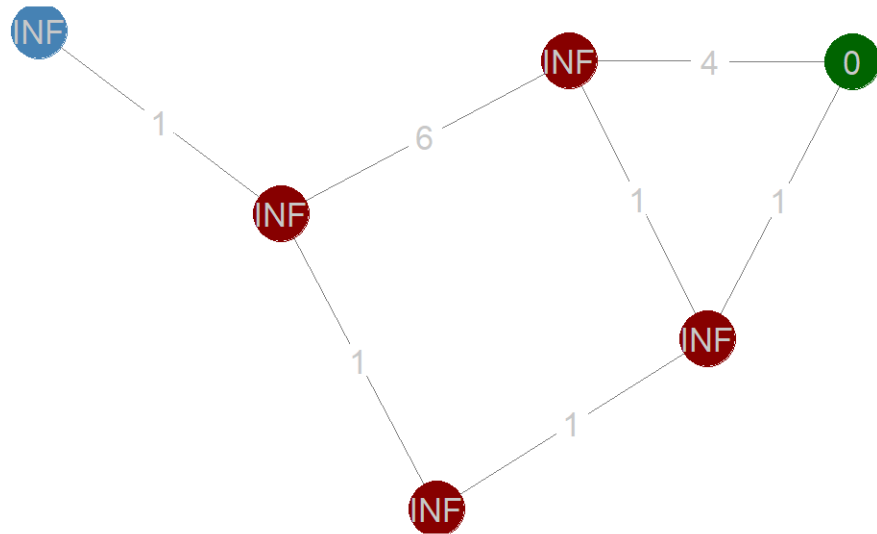
Oder sprachlich ausgedrückt

1. Setze Distanz des Startknoten auf 0 und die Distanz aller anderen Knoten auf unendlich
2. Alle Knoten "unbesucht" setzen
3. Aktueller Knoten ist Startknoten
4. Solange noch nicht alle Knoten "besucht" sind:
 1. Aktueller Knoten als "besucht" gesetzt.
 2. Für alle unbesuchten Nachbarknoten des aktuellen Knotens prüfen, ob Distanz eines Pfads über den aktuellen Knoten kleiner ist als die derzeitig hinterlegte Distanz des Nachbarknotens
 3. Wenn ja, aktualisiere Distanz des Nachbarn, so dass sie der Distanz vom Startknoten zum aktuellen Knoten plus der Distanz vom aktuellen Knoten zum Nachbarn entspricht
 4. Der nächste aktuelle Knoten ist ein unbesuchter Knoten mit geringster hinterlegter Distanz

Anmerkung 1: Wenn wir am Pfad zu **einem** Zielknoten interessiert sind, stoppen wir, wenn dieser als "besucht" gesetzt wird!

Anmerkung 2: Dijkstra funktioniert NICHT mit negativen Gewichten/Distanzen

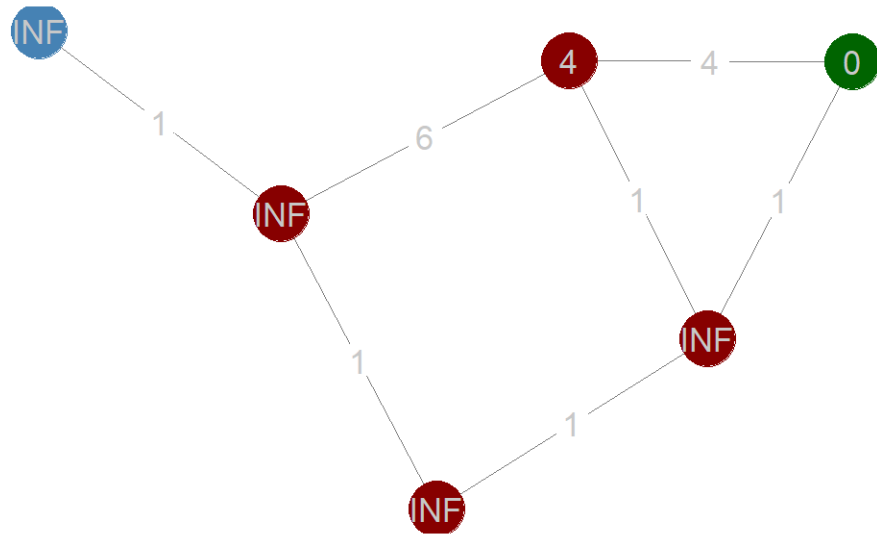
Dijkstra Visualisierung Schritt 1



Initialisierung: Distanz des Startknoten auf 0 setzen, ∞ für alle anderen. Startknoten ist aktueller Knoten.

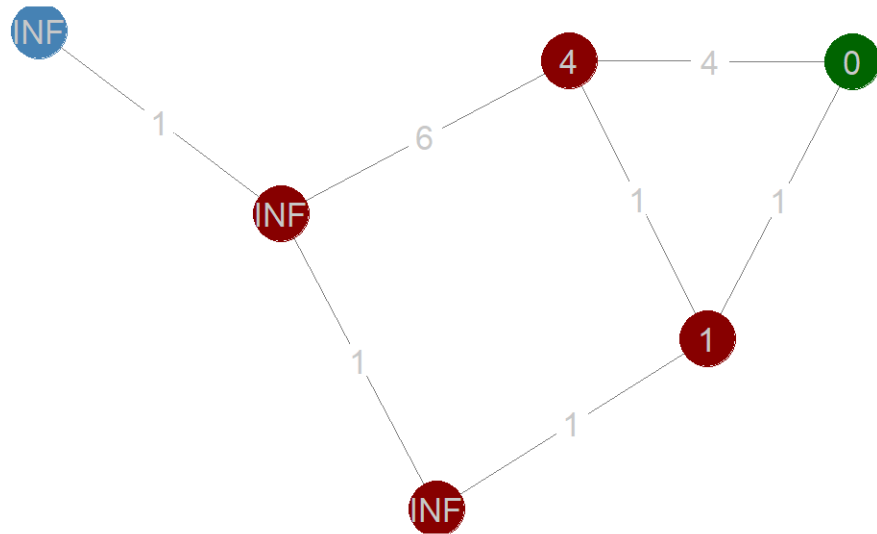
Grün - aktueller Knoten, *Rot* - unbesucht, *Blau* - Zielknoten.

Dijkstra Visualisierung Schritt 2.1



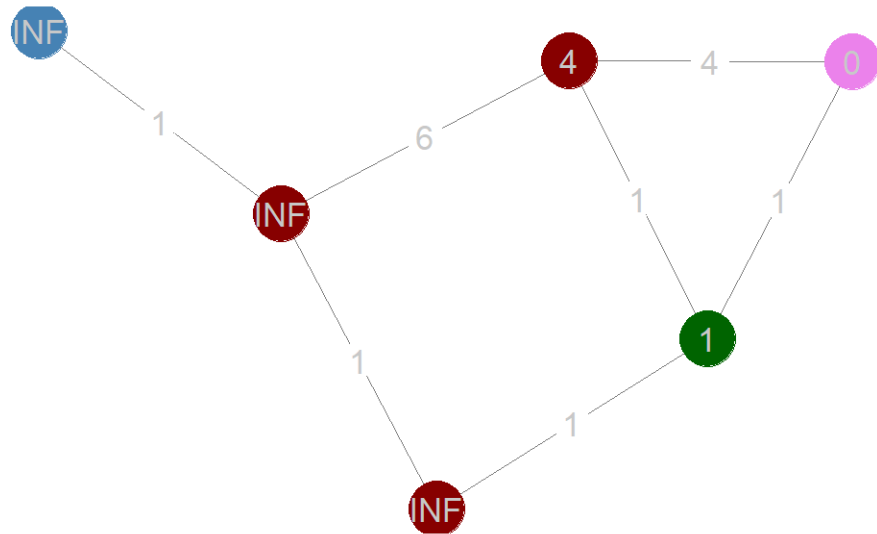
Update: Distanzen der Nachbarn des aktuellen Knotens werden gesetzt

Dijkstra Visualisierung Schritt 2.2



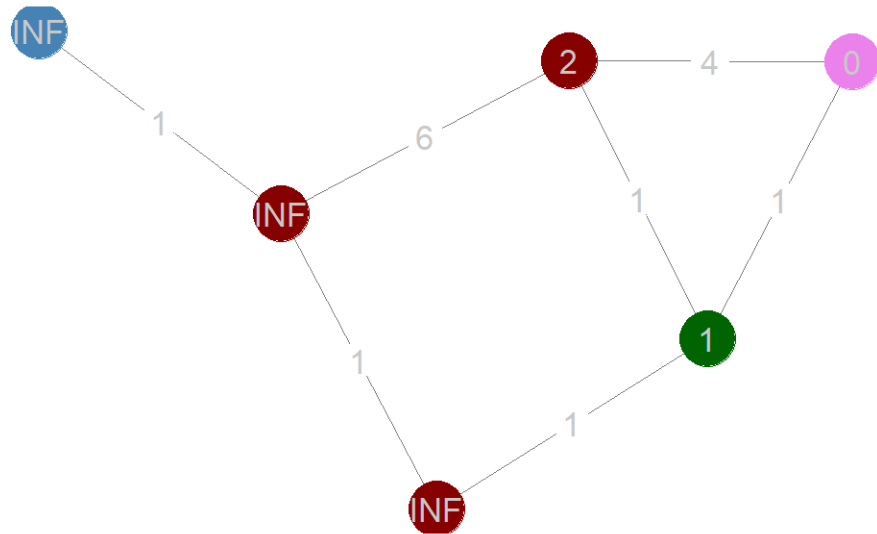
Update: Distanzen der Nachbarn des aktuellen Knotens werden gesetzt.

Dijkstra Visualisierung Schritt 3



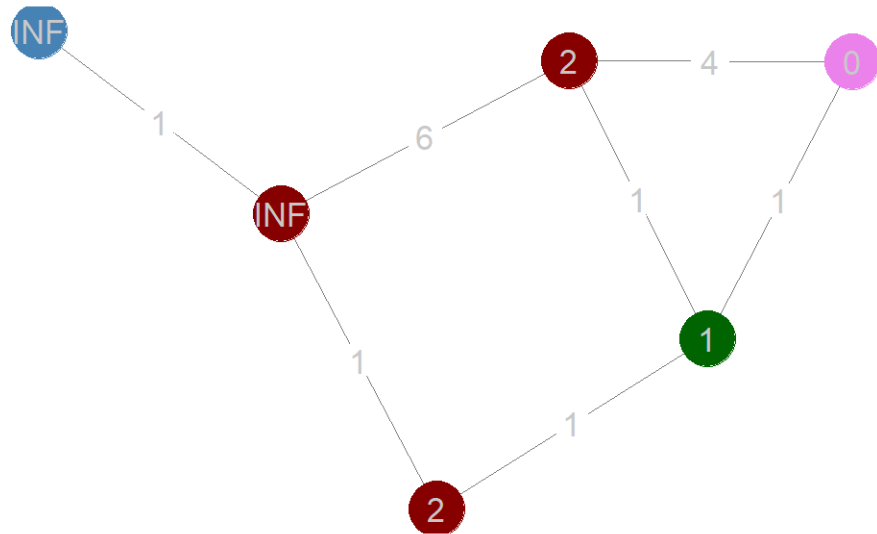
Knoten 1 wird als "besucht" gekennzeichnet (*Violett*). Knoten 2 hat geringste Distanz (1), wird aktueller Knoten.

Dijkstra Visualisierung Schritt 4.1



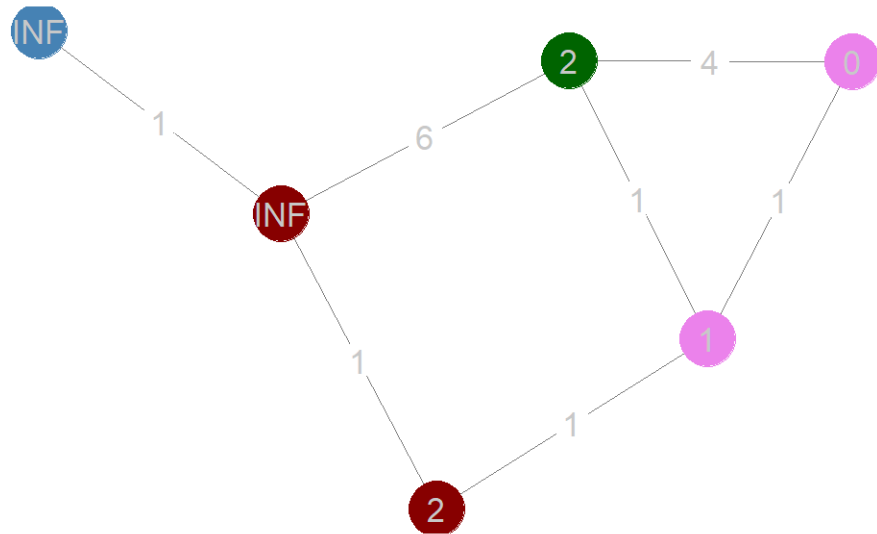
Update: Distanzen der Nachbarn des aktuellen Knotens werden gesetzt.

Dijkstra Visualisierung Schritt 4.2



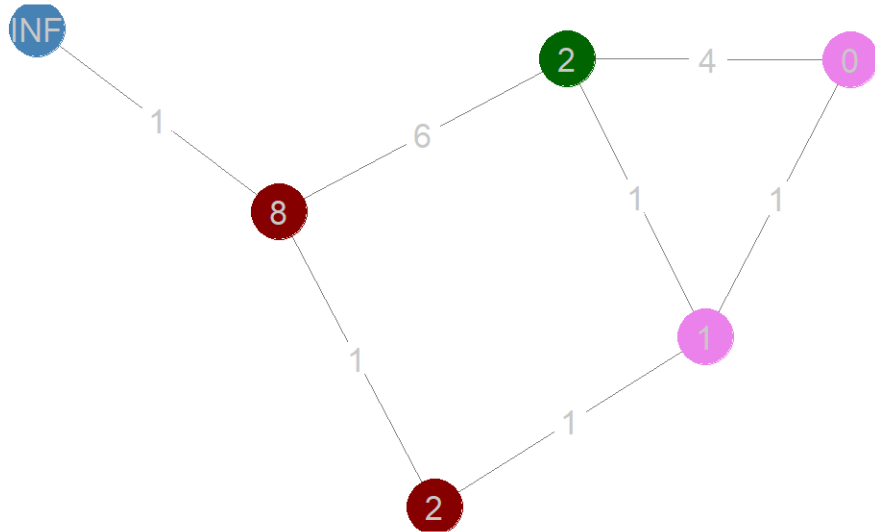
Update: Distanzen der Nachbarn des aktuellen Knotens werden gesetzt.

Dijkstra Visualisierung Schritt 5



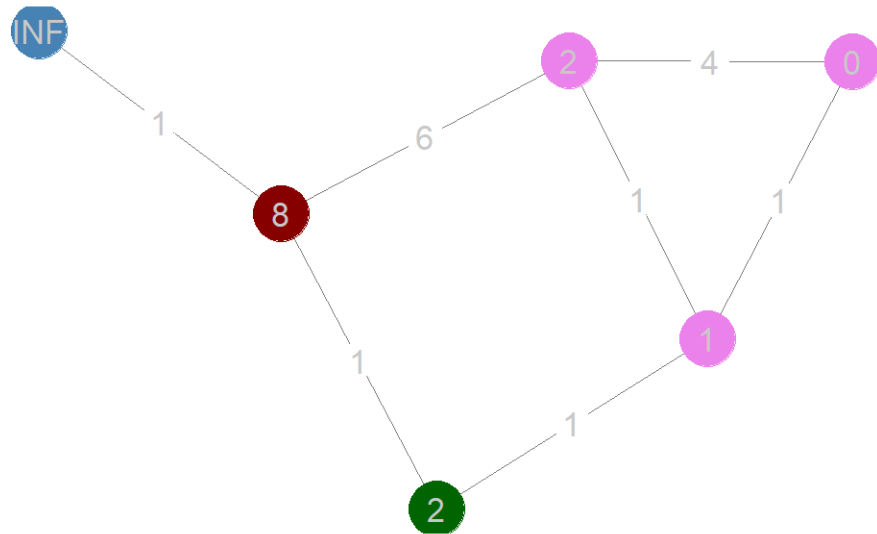
Knoten 2 wird als "besucht" gekennzeichnet. Knoten 5 hat geringste Distanz (2, identisch zu Knoten 3, beliebige Wahl), wird aktueller Knoten.

Dijkstra Visualisierung Schritt 6



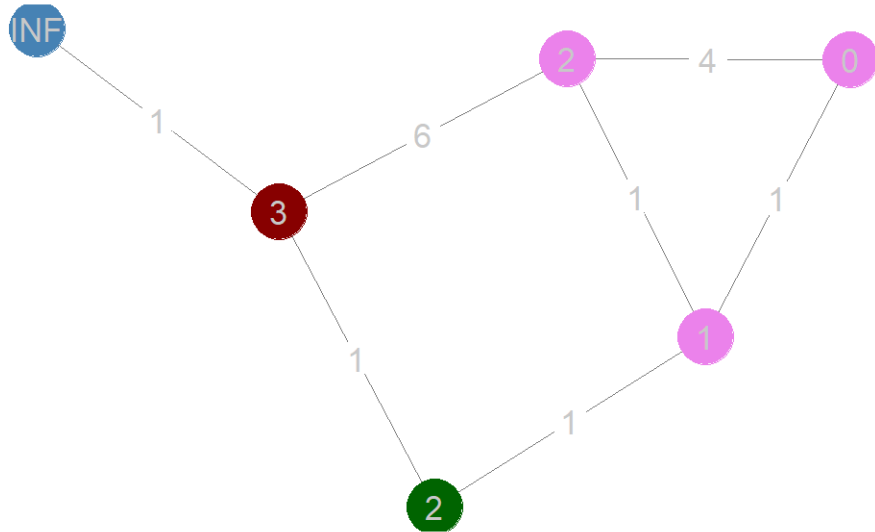
Update: Distanz des Nachbarn des aktuellen Knotens wird gesetzt.

Dijkstra Visualisierung Schritt 7



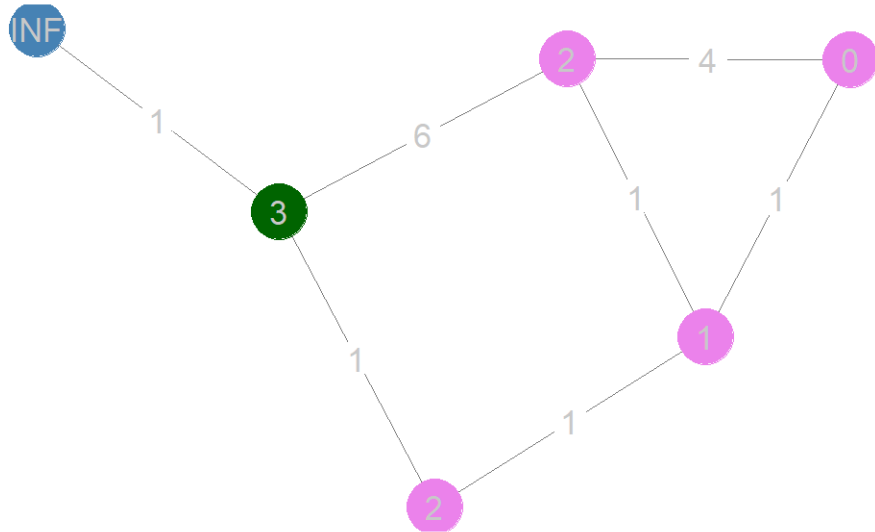
Knoten 5 wird als "besucht" gekennzeichnet. Knoten 3 hat geringste Distanz (2), wird aktueller Knoten.

Dijkstra Visualisierung Schritt 8



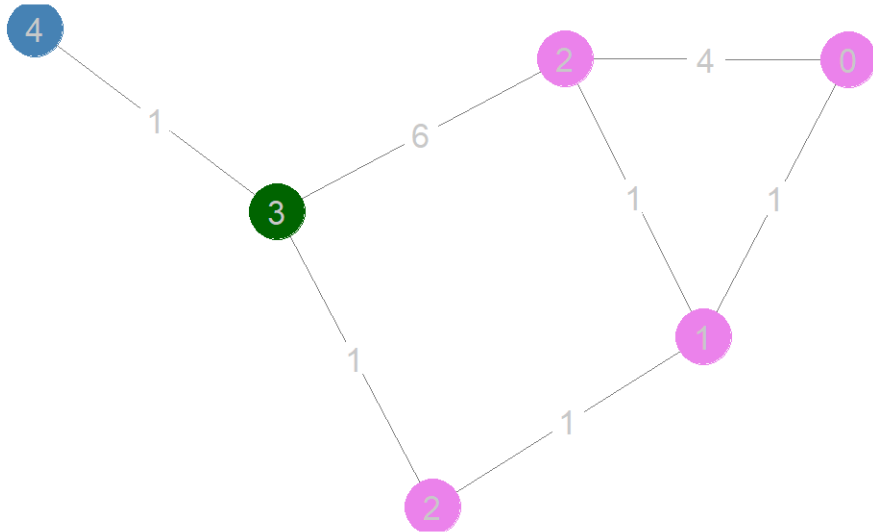
Update: Distanz des Nachbarn des aktuellen Knotens wird gesetzt.

Dijkstra Visualisierung Schritt 9



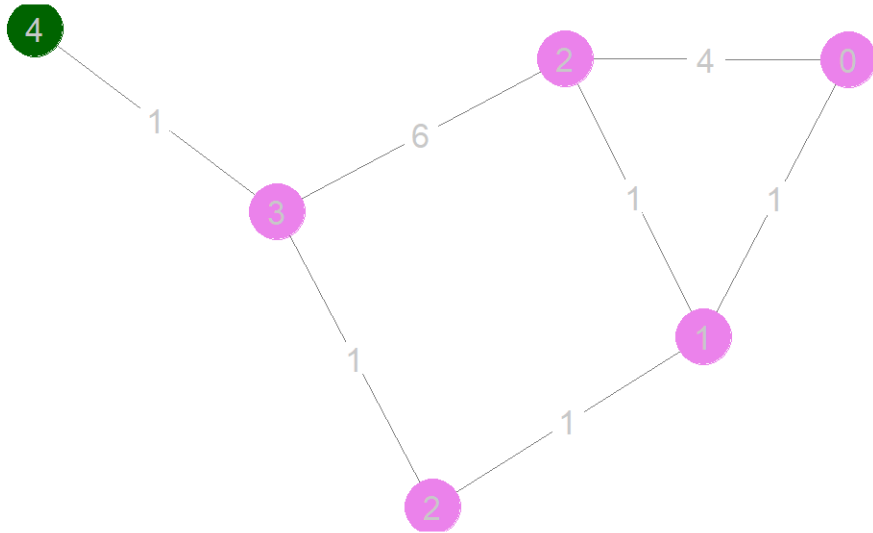
Knoten 3 wird als "besucht" gekennzeichnet. Knoten 4 hat geringste Distanz (3), wird aktueller Knoten.

Dijkstra Visualisierung Schritt 10



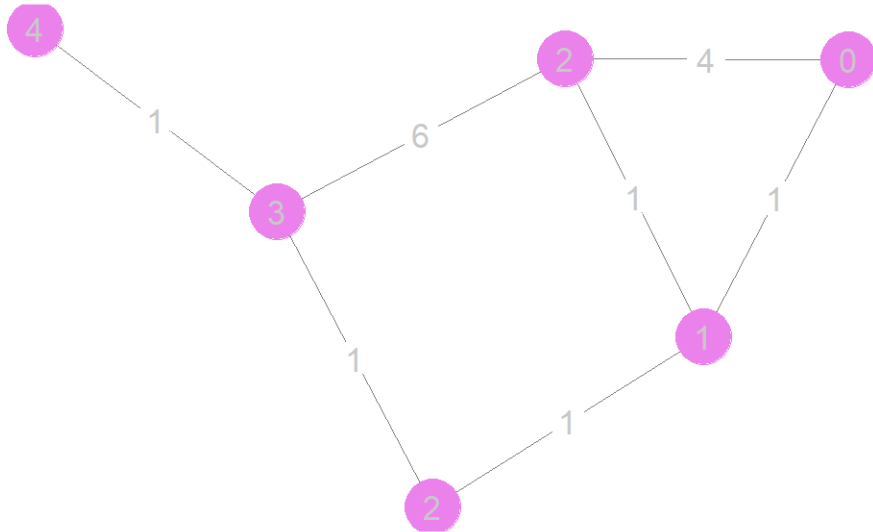
Update: Distanz des Nachbarn des aktuellen Knotens wird gesetzt.

Dijkstra Visualisierung Schritt 13



Knoten 4 wird als "besucht" gekennzeichnet. Knoten 3 hat geringste Distanz (3), wird aktueller Knoten.

Dijkstra Visualization Fertig



Letzter Knoten als besucht gekennzeichnet. Abgeschlossen!

Siehe andere Visualisierungen:

<https://www.cs.usfca.edu/~galles/visualization/Dijkstra.html>

R Anwendung

- siehe AnwendungOR-Dijkstra.R

A*

Andere Kürzester-Pfad Algorithmen

Dijkstra-Algorithmus bestimmt kürzeste Pfade zu allen anderen Knoten eines Graphs, ausgehend von einem Startknoten (wenn einzelner Zielknoten: früher abbrechen)

- **Dijkstra**: einzelner Startknoten, gerichtet und ungerichtet.
- **Bellman-Ford**: einzelner Startknoten, negative Gewichte möglich
- **A*** einzelner Startknoten + Zielknoten, mit Heuristik für Beschleunigung der Suche, gerichtet und ungerichtet
- **Floyd-Warshall algorithm** alle paarweisen kürzesten Pfade in gerichteten, dichten Graphen
- **Johnson** alle paarweisen kürzesten Pfade in gerichteten, spärlichen Graphen

Nachteil von Dijkstra

- Kein Zielknoten (bzw. Ziel nur für Abbruch genutzt)
- Ineffizient!

A*

- Ziel: Effizientere Suche nach kürzestem Pfad von Start- zu Zielknoten
- Funktioniert wie Dijkstra, aber:
 - Zusätzlich zur Distanz zum Ursprung wird eine Priorität gespeichert
 - Priorität: Distanz+Heuristik
 - Heuristik: Schätzung der "Distanz" (oder Kosten) zum Zielknoten
 - Beispiel bei Suchen auf einer Karte: Heuristik ist Distanz entlang Luftlinie von aktuellem Knoten zu Zielknoten
 - Der nächste Knoten wird anhand von Priorität gewählt, statt nur Distanz
- Nachteil A*: Falsche Heuristik (insbesondere Überschätzung der tatsächlichen Pfadlänge) kann zu nicht-optimalen Lösungen führen

A* - Initialisierung

- Graph G mit Knoten $G.V$, ein Startknoten $s \in G.V$, ein Zielknoten $z \in G.V$
- Distanz d , Status π , Vorgängerknoten α
- Priorität p , Heuristik $h(v, z)$

initialize(G, s, z, h)

for each vertex $v \in G.V$

$v.d = \infty$

$v.p = \infty$

$v.\pi = \text{unvisited}$

$v.\alpha = \text{none}$

$s.d = 0$

$s.p = h(s, z)$

A* - Update

- Aktueller Knoten u , Nachbarknoten v , Gewicht zwischen Knoten $w(u, v)$, Zielknoten z
- Distanz d , Status π , Vorgängerknoten α , Priorität p , Heuristik $h(v, z)$

update(u, v, w, z, h)

if $v.d > u.d + w(u, v)$

$v.d = u.d + w(u, v)$

$v.\alpha = u$

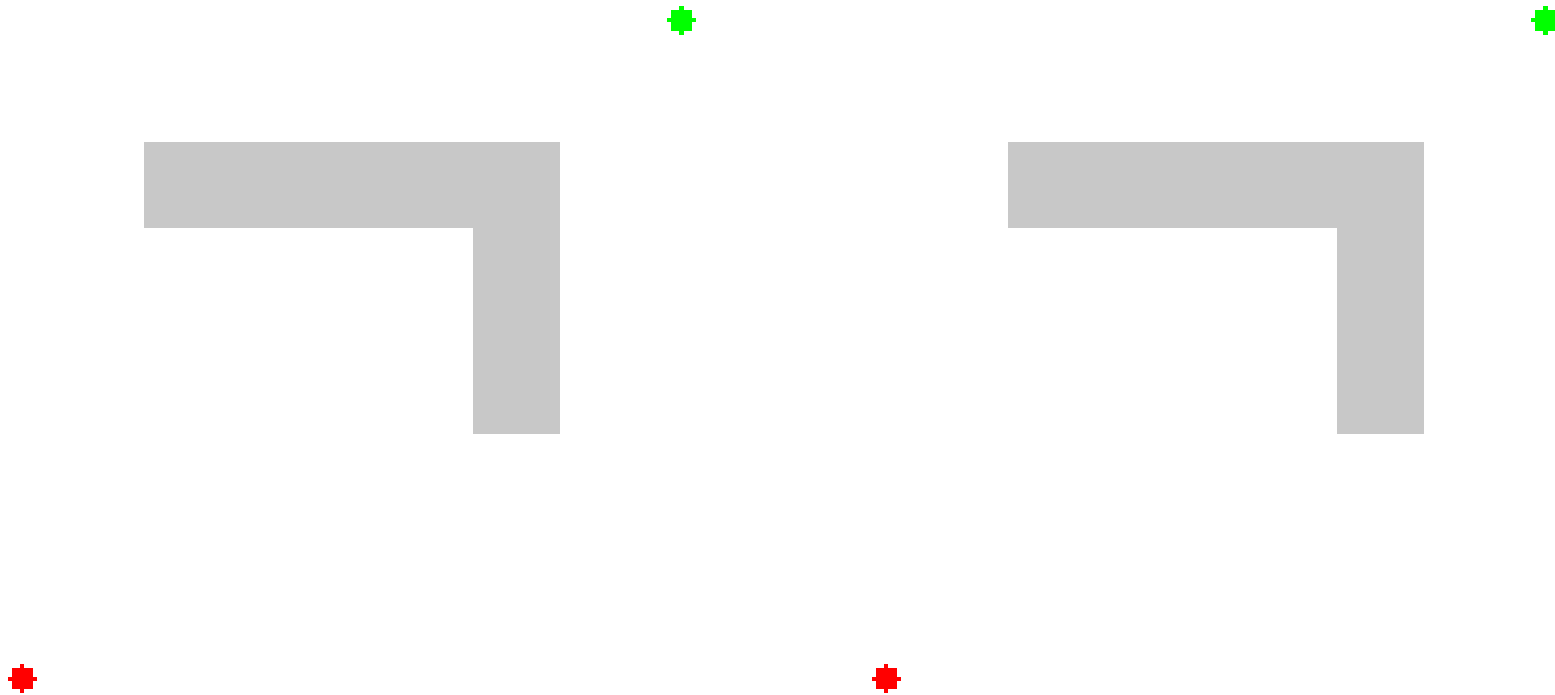
$v.p = v.d + h(v, z)$

A* - Algorithmus

- Graph G mit Knoten $G.V$, ein Startknoten $s \in G.V$, ein Zielknoten $z \in G.V$
- Distanz d , Status π , Vorgängerknoten α , Priorität p
- Aktueller Knoten u

```
A*(G, w, s)
  initialize(G, s, z, h)
  u = s
  while z.π == unvisited
    u.π = visited
    for each neighboring node v of u
      if v.π == unvisited
        update(u, v, w, z, h)
  u = find unvisited node with smallest u.p
```

Vergleich: Dijkstra vs. A*



[Subh83 / S. Bhattacharya, Wikimedia, CC BY 3.0](#) [Subh83 / S. Bhattacharya, Wikimedia, CC BY 3.0](#)

Übungsaufgaben

- 7.6 - 7.10 (Übung im moodle Kurse)

R Anwendung

- siehe AnwendungOR-Astar.R

TSP – Travelling Salesperson Problem

TSP – Travelling Salesperson Problem

Problem des Handlungsreisenden

- Gegeben: n Städte. Handlungsreisender muss:
 - Alle Städte genau ein Mal besuchen
 - Zur Ausgangsstadt zurückkehren
 - Kürzeste Tour?

TSP – Travelling Salesperson Problem

- Natürliche Repräsentation als Graph $G = (V, E, W)$
 - Städte V
 - Strecken E
 - Streckenlängen W

TSP – Anwendungsgebiete

- Anwendungsgebiete
 - Logistik (viele Erweiterungen und Varianten, z.B. VRP)
 - Design von Mikrochips
 - Layout integrierter Schaltkreise (Bohren von Leiterplatten)
 - Genom-Sequenzierung (Knoten = DNA-Teilstränge)

Ähnliche Probleme

- TSP ohne Rückkehr zum Startpunkt
- Vehicle Routing Problem (VRP) (mehrere 'Reisende', nicht nur ein Zyklus)
- Traveling Thief Problem (TSP + knapsack Problem)
- Stochastisch (Kosten, Distanzen sind mit Rauschen behaftet)
- Dynamische Varianten (Graph / Kosten / Distanzen ändern sich mit der Zeit)
- und viele, viele mehr.

Frage

- Wie würden Sie ein TSP lösen?

Brute force

- Brute force
 - Alle möglichen Touren ausprobieren
 - Einfacher Ansatz
 - Garantierte Lösung
 - Extrem hoher Aufwand

Approximative Lösung: Nearest Neighbour Heuristik

1. Start bei Startknoten
 2. Nächster Knoten: entlang Kante mit geringsten Kosten
- Wiederhole 2. bis alle Knoten besucht wurden.
 - Diese Heuristik ist ein "Greedy" Algorithmus - In jeder Iteration wird die beste Alternative gewählt

Nearest Neighbour Algorithmus

- Start in verschiedenen Städten liefert unterschiedliche Ergebnisse
- NNA Liefert in der Regel keine optimalen Lösungen
- Verbesserung des Verfahrens:
 - In jeder Stadt starten und bestes Ergebnis verwenden
 - Laufzeit erhöht sich allerdings um Faktor n (Anzahl Knoten)
 - Immernoch keine Garantie für gute Lösung

2-Opt Heuristik

- Verbesserung von approximativen Lösungen
 1. •Bestimme zuerst approximative Lösung (z.B. mit NNA)
 2. •Dann schrittweise verbessern
- Verbesserungsschritte:
 - Einen Teil der Tour invertieren (umgekehrte Reihenfolge der Knoten in der Tour)
 - Wähle jeweils die bessere Alternative (neue Tour oder alte Tour)
 - Nächster Tausch
- 2-Opt ist **Lokale** Suche!
 - Deshalb approximativ

Bessere Lösungen

- Modellierung als lineares Optimierungsproblem / Lineares Programmieren
- Evolutionäre Algorithmen

(behandeln wir hier nicht)

Übungsaufgaben

- 7.11, 7.12 (Übung im moodle Kurse)

R Anwendung

- Siehe AnwendungOR-TSP.R